

Northstar Care

DevSecOps, Azure och Defender för Cloud

Grupp 2 (Albin, Gustav, Valdemar och Vinh)

Med personlig fördjupning och reflektion skriven av Albin Sulejmani

Grupp 2 (Albin, Gustav, Valdemar och Vinh)	1
Applikationens struktur	3
Applikationens uppbyggnad	3
Arkitektur i Azure	3
Repots uppbyggnad	4
Motivering av reports uppbyggnad	4
Molnresurser och infrastruktur	6
Roller och rättigheter	6
Azure DevOps & Azure Portal	6
CI/CD-pipeline	7
Pipelinens steg och logik	7
Deployment	8
Secrets och konfigurationshantering	9
Säkerhetsanalys	10
Problem vi hittade:	10
Svaghet:	10
Hur vi prioriterade dem	13
Vad som upptäcktes i Defender, pipelinen och applikationen:	13
Pipeline: inget kritiskt säkerhetsproblem förutom 5 moderate severity sårbarheter som hittades av NPM-audit.	13
Defender:	14
För webbappen:	14
För northstar storage2 storage:	14
Säkerhetsmässiga styrkor:	15
Åtgärder	16
Vilka åtgärder som genomfördes:	16
Vilka problem som återstår:	17
Hur säkerhetsnivån förbättrades	18
Sammanfattning	19
Skapandet av resurser i Azure	19
Pipeline	19
Automatiserad säkerhetsgranskning	19
Datahantering och driftsäkerhet	19
Återstående problem	20
Slutsats	20
Personlig reflektion och fördjupning – Albin Sulejmani	21
Personlig fördjupning – Albin Sulejmani	22
Slutsats för personlig fördjupning	23

Applikationens struktur

Northstar Care-portalen är en fullstack webbapplikation avsedd för vård och interna ärenden inom Northstar Care. Systemet är uppbyggt med en trelagers struktur – frontend (React/Vite), backend (Express) och databas (SQLite). Det körs som en Node.js-process i Azure App Service.

Applikationens uppbyggnad

Systemet separerar ansvarsområden – frontend är utvecklad med React och Vite, backend drivs av Node.js och Express. En viktig detalj är hur dessa samverkar. I stället för att köra dem på separata servrar byggs frontenkoden till filer som placeras i backends wwwroot-mapp. Detta innebär att hela applikationen körs som en process i Azure. Detta minskar komplexiteten och förenklar hanteringen av CORS, vilket i längden gör att vi kan stärka säkerheten genom striktare brandväggsregler.

Arkitektur i Azure

1. Resursgrupp: Alla komponenter är samlade i en resursgrupp. Det underlättar säkerheten, kostnader, uppdateringar och livscykelhantering. I detta fallet underlättar det enormt eftersom vi kan isolera vårt projekt från andra grupper resurser i samma Azure miljö.
2. Web app: Vi skapade en ny resurs (web-app) i Azure portal. Vi kopplade den till *subscription1* och vår grupp *ITS25-Grupp2*, valde *Node 24 LTS* som runtime stack, och valde namnet *northstar-care-group-22* för vår webbapp.
3. Lagringskonto: Vi skapade ett lagringskonto "northstarstorage2". Den lagrar loggar och filer. I lagringskontot skapade vi en container som heter "documents" som vi vidare kopplade till vår web-app i miljövariablerna.
AZURE_STORAGE_CONNECTION_STRING = anslutnings sträng
AZURE_STORAGE_CONTAINER_NAME = documents
4. Azure DevOps och repo: Källkoden finns i ett git-repository i Azure DevOps. Det är kärnan i vår CI/CD-pipeline.

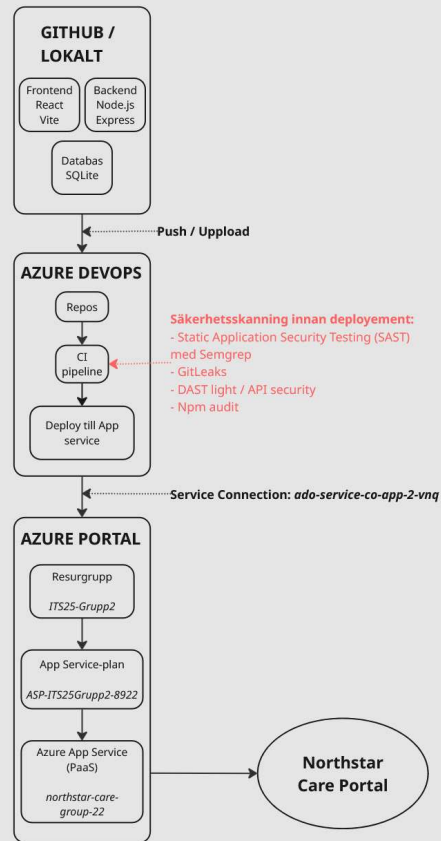
Repos uppbyggnad

```
/With-blob
├── .Pipelines-extrasec_test.yml #Automatiserad säkerhet och deployment
├── /Backend
│   ├── /server
│   │   ├── /storage          # Logik för filhantering
│   │   ├── /middleware      # Autentisering och indatavalidering
│   │   ├── /routes          # API-endpoints: Hanterar trafikvägar
│   │   ├── /services        # Applikationens interna funktioner
│   │   ├── index.js         # Integrerar applikationen
│   │   └── app.js           # Huvudapplikation och API-endpoints
│   ├── /wwwroot             # Destination för byggd frontend-kod
│   ├── /tests               # Automatiserade backend-tester
│   └── package.json         # Backend-beroenden
├── /Frontend
│   ├── /src                 # Frontend-källkod och komponenter
│   │   └── /app
│   │       └── api.js       # Centraliserad hantering av API-anrop till
│   │                               backenden
│   ├── auth.jsx            # Hanterar inloggning och sessioner
│   ├── /pages              # Sidstrukturer
│   ├── /tests              # Vitest-tester för frontenden
│   ├── vite-config.js      # Konfiguration för optimering
│   └── package.json        # Frontend-beroenden
├── .Pipelines-extrasec_test.yml # Pipeline
└── README.md
```

Motivering av reports uppbyggnad

- **Separation av miljöer:** Eftersom backend och frontend ligger i separata mappar kan vi köra specifika säkerhetsanalyser på varje del oberoende av varandra.
- **Byggnadsprocessen:** Pipelinen bygger frontenden i backend/wwwroot. Det minskar komplexiteten eftersom det bara behöver skickas ett paket till Azure app-service.
- **Modulär lagring:** Logiken för lagring är isolerad från resten av applikationen vilket gör det enkelt att byta mellan lokal lagring och Azure Blob Storage med hjälp av miljövariabler. Det gör det så man inte behöver ändra källkoden.

ARKITEKTUR WEBBAPP NORTHSTAR CARE OCH CI / CD FLÖDE



Molnresurser och infrastruktur

Vi kör Azure App Service (Linux) och medan vi laborerade upptäckte vi att vi hade valt fel resursnivå – när detta hade åtgärdats kvarstod problem med att kvoten överskreds. Vi märkte då att under hosting användes Free-planen istället för Basic (B1). När vi märkte detta och bytte, överskreds inte kvoten mer. Detta är viktigt då detta kan leda till att applikationen stängs ner och i vårdkontext kan detta innebära avbrott i tjänsteleveranser och få hårda konsekvenser för företag och patienter. Vidare så belyser detta komplexiteten i Azure som kan vara svår att navigera i. Fastän man tror att rätt resursnivå/grupp har valts, kan det finnas inställningar och val som överskrids.

Azure-resurser vi använde:

Resurs	Namn
Subscription	Azure Subscription 1
Resource Group	ITS25-Grupp2
App Service	northstar-care-group-22
App Service Plan	Basic B1 (Linux)
Storage Account	northstarstorage2
Blob Container	documents
Service Connection	ado-service-co-app-2-vnq

Roller och rättigheter

Hantering av identiteter och rättigheter var det största hindret i denna lab. Detta visar på de svårigheter som kan uppstå med identitetshantering i hybridmiljöer.

Azure DevOps & Azure Portal

Det finns en stor skillnad mellan rättigheter i Azure DevOps och Azure Portal. I Portal kontrollerar vi vem som får ändra på själva webbappen, repos och pipeline.

- **Identitetsproblematik** – Vi stötte på betydande hinder vid inloggning på grund av olika domäner (@learnnet.se och @azurenodehill.onmicrosoft.com). Vissa som loggade in genom sin Learnnet-domän fick inte tillgång till alla resurser och andra kunde inte logga in medan inloggningar genom Node Hill-domänen verkade fungera utan. Vidare så fick dem som loggade in med Node Hill-domänen contributor-behörigheter och dem med learnnet-domänen fick admin. Det gjorde att de med contributor-rollen inte kunde commita i main-branschen eller komma åt alla resurser. Detta visualiserade väldigt väl hur viktigt det är med rätt behörigheter och prenumerationer för att få tillgång till olika resurser på plattformen.

- **Service Connections** – För att fylla gapet mellan DevOps och Azure använde vi en Service Connection med namnet `ado-service-co-app-2-vnq`. Denna fungerar som en Service Principal. Istället för att enskilda användare behöver ha fulla adminrättigheter i Azure har vi gett pipelinen de rättigheter som krävs för att deploya koden. Detta följer principen om Least Privilege.

CI/CD-pipeline

Vår pipeline är definierad som kod i `azure-pipelines.yml`. Detta är en viktig del av DevSecOps – att säkerhetskontroller och tester är automatiserade och sker vid varje kodändring. Pipelinen inkluderar 4 säkerhetsskanningar som fungerar som gates och som därmed stoppar kod innan den når produktion. Dessa security gates skyddar tre nivåer: koden med SAST och secrets scanning, beroenden med npm audit, och leverans med security tests (DAST/API). Pipeline genererar också en artifact (*ci-report*) som innehåller säkerhetsresultaten samt frontend/backend testresultaten. Alla dessa resultat summeras i *ci-summary*-filen.

Pipelinenens steg och logik

1. **Miljöförberedelse** – Pipelinen startar en Ubuntu-agent och installerar rätt Node.js-version.
2. **Dependency Management** – Genom att köra `npm ci` kollar vi att exakt samma versioner av alla bibliotek installeras i pipelinen som de versioner som använts vid utveckling av koden. Detta förhindrar att nya sårbarheter tar sig in genom uppdaterade paket.
3. **Installerar och kör Static Application Security Testing (SAST) med Semgrep** – Testar och analyserar koden för att upptäcka möjliga injection-risker, XSS-problem och osäkra mönster.
4. **Installation och scanning av secrets i koden med Gitleaks** – Scannar repo och letar efter läckta API-nycklar, tokens, lösenord. Steget får inte stoppa pipelinen om hemligheter hittas (`|| true`) utan vi måste kolla *ci-report* vid upptäckta problem.
5. **Verify Hosting** – I vår pipeline startar backend-agenten med att köra en curl-förfrågan mot `/health`. Om appen inte kan starta lokalt i pipelinen så avbryts deploy processen. Detta förhindrar att en fungerande webbapplikation tas ner med dålig kod.
6. **Run API security och DAST light tests** – Testet bör köras mot en körande applikation och därför efter steg 3 Verify Hosting. Testet letar efter auth-problem, öppna endpoints och säkerhetsbrister i API:et. Det kontrollerar också security headers (Missing X-Frame-Options och CSP).
7. **Automatiserade tester (Vitest)** – Vi kör frontend och backend tester med Vitest och exporterar rapporten med JUnit, exempel nedan.

```
# Kör frontend-tester med Vitest och exporterar JUnit-rapport.
- script: npx vitest run --reporter=junit
  --outputFile="$(reportDir)/frontend-junit.xml"
  displayName: "Run frontend tests"
  workingDirectory: "$(Build.SourcesDirectory)/Frontend"
```

Sedan publiceras JUnit-rapporterna direkt i webbgränssnittet med koden nedan

```
# Publicerar frontendens JUnit-resultat även om tidigare steg har misslyckats.
- task: PublishTestResults@2
  displayName: "Publish frontend test results"
  condition: succeededOrFailed()
  inputs:
    testResultsFormat: "JUnit"
    testResultsFiles: "$(reportDir)/frontend-junit.xml"
    testRunTitle: "Frontend Vitest"
    failTaskOnFailedTests: true
```

Detta gör att man direkt kan se vilka moduler som brister utan att behöva leta rätt på loggfiler, vilket i sin tur ökar "Mean Time To Repair" (MTTR) eftersom vi snabbt kan identifiera om någon funktionalitet har gått sönder.

8. **Sårbarhetsskanning (npm audit)** – Alla tredjepartsbibliotek analyseras efter kända sårbarheter genom npm audit.
9. **Deploy** – Sker via AzureWebApp med Service Connection *ado-service-co-app-2-vnq* mot *northstar-care-group-22*. Deployment körs enbart när de föregående stegen har passerat.
10. **Report build status** - Gör en rapport för vad som gjorts

Deployment

1. **Paketering:** Pipelinen tar innehållet i backenden (som även innehåller den färdigbyggda frontenden i *wwwroot*-mappen) och skapar en zip-fil.
2. **Verifiering:** Skriptet letar efter specifika filer såsom *AzureBlogStorage.js* och kollar så att rätt bibliotek finns i *package.json*.
3. **Driftsättning:** Pipelinen använder den service connection vi angett (*ado-service-co-app-2-vnq*) som ett handshake för att få behörigheter att ändra i resursgruppen. Sen laddas zipfilen upp som skapades i början som *app.zip* i *web-appen*.
4. **Omstart:** Slutligen för att verkställa den nya koden startar *node.js*-servern så att den nya koden blir tillgänglig för alla användare.

Secrets och konfigurationshantering

Säker hantering av hemligheter är särskilt viktig i molnmiljö. Vi har i vår pipeline gått från att bara dölja data till att skanna efter den.

AZURE_STORAGE_CONNECTION_STRING	DefaultEndpointsProtocol=https;AccountName=northstarstorage;
AZURE_STORAGE_CONTAINER_NAME	documents
SCM_DO_BUILD_DURING_DEPLOYMENT	true
STORAGE_TYPE	azure

När vi granskade koden hittade vi i server/index.js, server/config/runtimePaths.js och server/app.js olika konfigurationsbehov. Förutom de variabler vi lagt in i tabellen nedan hanterade vi även SCM_DO_BUILD_DURING_DEPLOYMENT, vilket är en miljövariabel som talar om för Azure App Service att den ska bygga koden och köra nödvändiga skript som npm install vid driftsättningen.

Variabel	Känslighetsnivå	Syfte
JWT_SECRET	Hemlig	Signerar och verifierar JWT-token. Kritisk då ett känt värde ger full kontroll över autentisering
AZURE_STORAGE_CONNECTION_STRING	Hemlig	Autentisering mot Azure Blob Storage. Innehåller kontonyckel
AZURE_STORAGE_CONTAINER_NAME	Konfiguration	Namn på Blob container (documents)
STORAGE_TYPE	Konfiguration	Väljer mellan local och azure som lagringsbackend
NODE_ENV	Konfiguration	En miljövariabel som talar om för appen om den körs i t.ex. development eller production. Används för att ändra betende t.ex mer information i debug eller mer säker körning i prodouction
PORT	Konfiguration	Sätts automatiskt av App Service så behövs inte konfigureras manuellt

Hantering av JWT_SECRET

En kritisk och känslig del i konfigurationen är som sagt JWT_SECRET. I källkoden (server/index.js) upptäckte vi att applikationen använder en hårdkodad fallback värde kallat "northstar-dev-secret". Eftersom detta är tydligt synligt i Git-historiken utgör det en allvarlig säkerhetsbrist. Värdet kan användas av vem som helst och det kan förfälska JWT-token för att få åtkomst.

Den korrekta åtgärden är att skapa en unik JWT_SECRET i app service application settings och sedan radera fallbackvärdet ur källkoden så att applikationen vägrar starta om nyckeln saknas.

Hur vi valde att lagra

AZURE_STORAGE_CONNECTION_STRING, AZURE_STORAGE_CONTAINER_NAME och STORAGE_TYPE la vi in som App Service Application Settings i Azure Portal. Värdena hämtades från Storage Account northstarstorage2 under Säkerhet och nätverk → Åtkomstnycklar – Anslutningssträng. På det sättet injiceras de som miljövariabler vid körning och syns inte i källkoden eller pipeline loggarna.

Säkerhetsanalys

Problem vi hittade:

Svaghet:

- *authController.js* verkar inte ha något rate-limiting för lösenord och är då öppen för brute-force-attacker.

```
const result = authService.login({ username, password });
if (!result) {
  return res.status(401).json({ error: "Invalid credentials" });
}
```

- Samma sak i *auth.js*: det finns ingen rate-limiting för token vilket kan innebära en risk för brute-force attack mot tokens.

```
const decoded = jwt.verify(token, jwtSecret);
if (decoded.jti && authRepository.isTokenRevoked(decoded.jti)) {
  return res.status(401).json({ error: "Invalid token" });
}
```

- Vid genomgång av källkoden i backend > *assistant.routes.js* identifierade vi att endpointen för *assistant* definieras som:

```
router.post("/ask", authMiddleware, assistantController.ask);
```

Endpointen är skyddad av autentisering via *authMiddleware* men vi såg ingen input validering på route- eller controller-nivå.

Vid vidare analys i *controllers > assistantController.js* observerade vi följande implementation:

```
const question = req.body?.question;
const result = assistantService.ask({ question, user: req.user });
```

Detta visar att ingen explicit input validering av *question* sker och att användarinput skickas direkt vidare till service-lagret utan filtrering. Ogiltig eller extrem input kan nå backend-logik vilket vi testade via direkt anrop av endpointen */api/assistant/ask* via fetch i webbläsar konsolen med en payload av 100 000 tecken:

```
> fetch("/api/assistant/ask", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer
  },
  body: JSON.stringify({
    question: "A".repeat(100000)
  })
})
.then(res => res.json())
.then(data => console.log(data));
< Promise {<pending>}
VM4055:12
{answerId: 'ans-1776819473827-88322', answer: 'Question: AAAAAAAAAA
AAAAAAAAAAAAAAAAAAAAAAAAAAAA...rge Summary 2026-04-05, Clinical Not
e 2026-04-09.', sources: Array(4), permissionMatches: Array(0),
mode: 'enabled'}
```

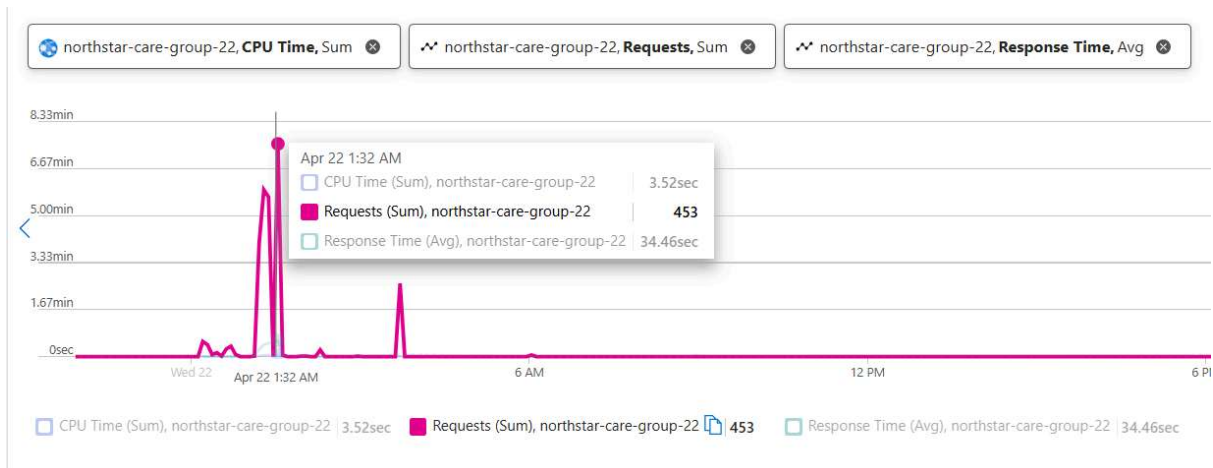
Requesten accepterades och returnerade ett giltigt svar, men vid en payload på 1 000 000 tecken returnerade servern en HTTP 500 Internal Server Error. Detta tyder på att begränsning av inputstorlek inte hanteras tidigt i requestkedjan, vilket kan leda till onödig resursförbrukning och instabilitet vid extrem input, förutsatt att kontot är hackat och missbrukas.

Ytterligare testade vi systemets robusthet genom ett belastningstest med k6: vi skickade ~400 requests under 30 sek via ett skript. Vid analys av Azure App Service metrics (i Monitoring) observerade vi:

Response time: 34,46 sek.

CPU usage: låg men hög latency, 3,52 sek (CPU time).

Totalt antal requests: 453.



Resultatet visade att systemet inte kraschade under belastningen, men att latens ökar kraftigt, vilket tyder på begränsad skalbarhet.

Sammantaget innebär avsaknad av tidig inputvalidering i kombination med avsaknad av tydliga begränsningar (t.ex. max payload-storlek och rate-limiting) en risk för ökad resursförbrukning. Vid missbruk av autentiserade konton kan detta bidra till prestanda degradering och DoS-liknande beteende. Detta prioriterade vi dock inte eftersom vi anser att risken var låg/medelhög.

-> Förslag på åtgärder: att lägga till input-validering och rate-limiting direkt i källkoden och skapa en New alert rule i Azure Monitoring som triggas vid ovanligt många requests för att övervaka en sådan incident.

- Vi testade om man kunde manipulera skapandet av ett ärende. Genom att följa flödet med samma identifieringsprocessen (routes -> controller -> services) tittade vi i källkoden i backend >casesService.js:

```
createCase: ({ payload, user }) => {
  const created = casesRepository.create({
    externalRef: payload.externalRef || nextExternalRef(),
    title: payload.title,
    description: payload.description,
    status: payload.status || "open",
    priority: payload.priority || "medium",
    assignedUserId: payload.assignedUserId ?? user.id,
    team: payload.team || "support"
  });
};
```

Där identifierade vi en potentiell trust boundary violation.

Genom att testa via direkt anrop av endpointen `/api/cases` via fetch i webbläsar konsolen och inloggad som anna.support:

```

> fetch("https://northstar-care-group-22.azurewebsites.net/api/cases", {
  method: "POST",
  headers: {
    "Content-Type": "application/json",
    "Authorization": "Bearer
  },
  body: JSON.stringify({
    title: "hellooo",
    description: "test 3",
    status: "open",
    priority: "critical",
    assignedUserId: 4,
    team: "manager"
  })
})
.then(res => res.json())
.then(data => console.log(data))
.catch(err => console.error(err));
< Promise {<pending>}
VM306:17
{id: 39, external_ref: 'NS-2026-491516', title: 'hellooo', description: 'test 3', status: 'open', ...}

```

Requesten accepterades och returnerade ett giltigt svar. Caset skapades av anna-support som bara mikael-manager kan se, vilket inte borde vara möjligt.

Hur vi prioriterade dem

Vi prioriterade först problemen som upptäcktes av pipelinen. Sedan de kritiska problemen som upptäcktes av Microsoft Defender.

Vad som upptäcktes i Defender, pipelinen och applikationen:

Pipeline: inget kritiskt säkerhetsproblem förutom 5 moderate severity sårbarheter som hittades av NPM-audit.

```

# CI Report

## Overall Status
🚩 PIPELINE PASSED WITH WARNINGS

## Build and Hosting
- Frontend build completed and emitted into Backend/wwwroot.
- Backend startup and hosting checks passed for /health and /.

## Tests
- Frontend JUnit: frontend-junit.xml
- Backend JUnit: backend-junit.xml

## Dependency Scan (npm audit)
| Area | Low | Moderate | High | Critical | Total |
|----|----|----|----|----|
| Backend | 0 | 5 | 0 | 0 | 5 |
| Frontend | 0 | 5 | 0 | 0 | 5 |

## Security
- SAST (Semgrep): ✓ PASS (0 issues, 0 medium/high)
- Secrets (GitLeaks): ✓ PASS (0 findings)
- Security Headers: WARNING MISSING
- DAST: ✓ PASS

```

Genom att köra *npm audit* lokalt i konsolen i *./Backend* och i *./Frontend* fick vi fler detaljer och rekommendationer: de sårbara Devdependencies Vite och Vitest behövde uppdateras.

```
# npm audit report

esbuild <=0.24.2
Severity: moderate
esbuild enables any website to send any requests to the development server and read the response - https://github.com/a
dvisories/GHSA-67mh-4wv8-2f99
fix available via `npm audit fix --force`
Will install vitest@4.1.4, which is a breaking change
node_modules/esbuild
  vite <=6.4.1
  Depends on vulnerable versions of esbuild
node_modules/vite
  @vitest/mocker <=3.0.0-beta.4
  Depends on vulnerable versions of vite
node_modules/@vitest/mocker
  vitest 0.0.1 - 0.0.12 || 0.0.29 - 0.0.122 || 0.3.3 - 3.0.0-beta.4
  Depends on vulnerable versions of @vitest/mocker
  Depends on vulnerable versions of vite
  Depends on vulnerable versions of vite-node
node_modules/vitest
  vite-node <=2.2.0-beta.2
  Depends on vulnerable versions of vite
node_modules/vite-node

5 moderate severity vulnerabilities

To address all issues (including breaking changes), run:
npm audit fix --force
mrvinh@MSI:/mnt/c/Users/thuyt/Desktop/northstar-care-portal-main/Backend$
```

Defender:

För webbappen:

En kritisk och en låg risk för anonymous access och exposure to the internet upptäcktes.

Defender rekommenderar att:

- Kritisk: *Managed identity should be enabled on web apps* för att minska risken av komprometterade hemligheter och för att öka säkerheten vid åtkomst till Azure-resurser.
- Låg: *Diagnostic logs in App Service should be enabled* för att utreda säkerhetsincidenter eller nätverks kompromettering.

För northstar storage2 storage:

En hög risk och två låga risker för exposure to the internet upptäcktes. Defender rekommenderar att:

- Hög: den delade nyckeln bör inaktiveras och att endast tillåta Azure AD att auktorisera tillgång till vårt lagringskonto.
- Låg: implementera säker kommunikation med lagringskontot genom att lägga till en privat endpoint: vi bör använda en privat anslutningslänk för vårt lagringskonto för att minska risken för dataintrång eller obehörig åtkomst.
- Låg: storage accounts should restrict network access using virtual network rules. Just nu är vårt lagringskonto publikt exponerat vilket innebär en risk för kompromettering av datasäkerheten.

Identifierade problem:

1. Credentials leaks: Risken att secrets hamnar i källkoden hanteras nu via Git Leaks.
2. Tillgänglighetsrisker (Availability/Flood, etc.): Problem med resurskvoter när vi använde "Free"-planen visade på ett stort availability-problem då vi endast kunde ha tjänsten tillgänglig under ca 60 minuter per dag.
3. Beroenderisker: Vår npm audit visade att vi har beroenden med kända sårbarheter. Detta är en ständig risk när vi förlitar oss på kod skriven av andra personer. Vi valde på grund av tidsbrist och ifall något skulle sluta fungera att inte fixa de 5 beroenden som gav 'moderate' sårbarheter, men detta är något som bör åtgärdas inom tid.
4. Konfigurationsfel: Svårigheter med att få Azure Blob Storage att fungera visade risker med manuell konfiguration.

Säkerhetsmässiga styrkor:

1. Token Revocation: Trots avsaknad av rate limiting i form av att koden kör "authService.login" och om resultatet är falskt returneras omedelbart en 401-status. En viktig styrka är att koden innehåller ett sätt att kontrollera om nyckeln har spärrats "isTokenRevoked" vilket gör det möjligt att omedelbart kunna göra stulna nycklar obrukbara vid utloggning.

```
const decoded = jwt.verify(token, jwtSecret);  
  
if (decoded.jti && authRepository.isTokenRevoked(decoded.jti)) {  
  return res.status(401).json({ error: "Invalid token" });  
}
```

Här på bilden kan man se att koden kontrollerar om sessionsnyckeln har spärrats.

2. Vi integrerade även fyra automatiska säkerhetstester i pipelinen som var SAST, Secrets scanning, DAST och Dependency scanning. Detta skapar ett starkt filter som stoppar sårbarheter innan de kan nå molnet.

```
# CI Report  
  
## Overall Status  
🚩 PIPELINE PASSED WITH WARNINGS  
  
## Build and Hosting  
- Frontend build completed and emitted into Backend/wwwroot.  
- Backend startup and hosting checks passed for /health and /.  
  
## Tests  
- Frontend JUnit: frontend-junit.xml  
- Backend JUnit: backend-junit.xml  
  
## Dependency Scan (npm audit)  
| Area | Low | Moderate | High | Critical | Total |  
|-----|-----|-----|-----|  
| Backend | 0 | 5 | 0 | 0 | 5 |  
| Frontend | 0 | 5 | 0 | 0 | 5 |  
  
## Security  
- SAST (Semgrep): ✓ PASS (0 issues, 0 medium/high)  
- Secrets (GitLeaks): ✓ PASS (0 findings)  
- Security Headers: WARNING MISSING  
- DAST: ✓ PASS
```

3. Vi tog även bort behovet av att spara känsliga lösenord för interna Azure-tjänster genom att aktivera systemtilldelad identitet. Detta gör att webbappen endast har de rättigheter den måste ha. Least privilege principen. Det aktiverades i Inställningar Identitet -> Systemtilldelad.

Åtgärder

Vilka åtgärder som genomfördes:

- Uppdatering av Backend:
Updating vitest to 4.1.5, which is a SemVer major change.
added 14 packages, removed 17 packages, changed 16 packages, and audited 216 packages in 13s
- Och Frontend:
npm warn audit Updating vite to 8.0.9, which is a SemVer major change.
npm warn audit Updating vitest to 4.1.5, which is a SemVer major change.
added 176 packages, and audited 177 packages in 6s
- Test: Webbappen fungerar lokalt.
- Hanterad identitet bör användas i webbappar. Vi aktiverade detta för *northstar-care-group-22* i inställningar → Identitet → Systemtilldelad
- Vi skapade en diagnostic setting "Loggar-Grupp 2" i Monitoring → Diagnostic settings → loggar App Service Application Logs, Access Audit Logs, IPSecurity Audit logs, App Service Authentication logs (preview) och AllMetrics i vårt lagringskontot (vi saknade behörigheter för att skapa en workspace i *Application Insights* och koppla loggarna till den).

Extra säkerhet:

- Hanteringen av data retention i Blob för att minska risken att känslig data lagras för länge → detta hanterades genom att lägga till en regel i Azure Lifecycle Management för vår lagring *northstarstorage2*. Gamla dokument raderas om ingen modifiering sker inom 365 dagar → detta för att minska exponering vid dataläckor och för att följa data minimeringsprincipen.

Update a rule

Details Base blobs

Lifecycle management uses your rules to automatically move blobs to cooler tiers or to delete them. If you create multiple rules, the associated actions must be implemented in tier order (from hot to cool storage, then archive, then deletion).



- Aktivering av Redundancy för lagringskontot: (Geo-redundant storage (GRS)) för att skydda mot tillfälliga hårdvarufel, nätverks- eller strömavbrott och naturkatastrofer. Vi kunde däremot inte skapa en backup-vault BackupBlobGrupp2 med geo-redundant och Soft Delete Retention Period of 14 dagar för att Enable Azure Backup for blobs på grund av behörigheter.

Vilka problem som återstår:

Ett kritiskt fynd hittades i server/[index.js](#) som innehöll följande rad kod:

```
const JWT_SECRET= process.env.JWT_SECRET || "northstar-dev-secret";
```

Standard eller fallbackvärdet "northstar-dev-secret" är ett hårdkodat standardlösenord som syns publikt, det ligger nu i Git historiken. Om JWT_SECRET inte sätts direkt i App Service konfigurationen så körs applikationen med detta kända värde, vilket innebär att vem som helst kan förfalska JWT-tokens och logga in som användare, inklusive admin.

Det bästa sättet att hantera detta hade varit att dels sätta JWT_SECRET som en Application Setting i App Service, och dels ta bort fallbackvärdet ur koden så att applikationen inte startar om variabeln saknas. Detta är något vi inte ändrade men som bör åtgärdas innan applikationen sätts i bruk.

Anledningen till att hemligheter inte ska ligga i report är på grund av att Git historiken är permanent. Även om ett värde tas bort i en ny commit, finns det kvar i historiken och man kan enkelt hitta det igen.

- Vi avaktiverade Allow storage account key access i northstarstorage2 → Settings → Configuration, och lade till en ny roll [Storage Blob Data Contributor](#) för Grupp 2 i Access Control (IAM) men webbappen tappade tillgången till lagrings kontot för att t.ex ladda upp dokument eftersom källkoden behöver uppdateras genom att ta bort connection-string till Blob:

```
const azureConnectionString = env.AZURE_STORAGE_CONNECTION_STRING ||  
"";
```

Vi insåg att tid och djupare programmeringskunskaper behövdes för att utföra detta och valde att avvakta.

- Begränsning av nätverksaccess till vårt lagringskonto genom att skapa en virtuell nätverksregel och lägga till en privat endpoint.
- Aktivering av Microsoft Defender for Storage för att detektera hot mot känsliga data och uploading av malware scanning. Detta innebär dock en extra kostnad.

Hur säkerhetsnivån förbättrades

Eftersom vi har implementerat en DevSecOps-pipeline och flyttat applikationen till Azure har vi gått från en reaktiv säkerhet till en proaktiv och automatiserad process. Säkerhetsnivån har förbättrats mycket inom följande områden:

1. Automatisering av säkerhetskontroller (Security Gates)

Den största förbättringen är de automatiserade "gates" i vår CI/CD-pipeline. Vi har gått från att dölja känslig data till att aktivt skanna efter den.

- **SAST (Static Application Security Testing):** Med hjälp av Semgrep analyseras koden vid varje push för att identifiera infektionsrisker, XSS-problem och osäkra kodmönster.
- **Secrets Scanning:** Genom Gitleaks säkerställer vi att inga API-nycklar eller lösenord läcker genom källkoden.
- **Beroendehantering:** Vi använder npm audit för att identifiera sårbarheter i tredjepartsbibliotek. Vi har bland annat åtgärdat sårbarheter genom att uppdatera centrala verktyg som Cite och Vitest.
- **DAST Light & API Security:** Vi testade applikationen mot vanliga hot och verifierade att säkerhetsheader som CSP och X-Frame-Options är korrekt konfigurerade för att skydda användarna.

2. Infrastruktur och identitetshantering

Säkerheten i molnmiljö har stärkts genom att följa standard principer för molnsäkerhet:

- **Managed Identity:** Vi har aktiverat systemtilldelad identitet för webbappen, vilket minskar behovet av att hantera och lagra hemligheter manuellt och därmed sänker risken för att de hamnar fel.
- **Least Privilege:** Genom att använda en Service Connection (Service Principal) mellan Azure DevOps och Azure Portal behöver enskilda utvecklare inte ha administrativa rättigheter. Pipelinen har endast de rättigheter som krävs för att utföra deployment.

3. Tillgänglighet och Övervakning (Resilience)

En viktig del av säkerheten är tillgängligheten (Availability)

- **Övergång från Free till Basic:** Genom att byta från Free-planen till Basic B1 eliminerade vi risken för tjänsteavbrott på grund av kvotbegränsningar, vilket är kritiskt i just denna app inom vårdkontext.
- **Logging och Incident Response:** Vi har skapat en diagnostisk inställning (Loggar-Grupp 2) som skickar auditloggar och autentiseringsloggar till vårt lagringskonto. Detta gör det enkelt att i efterhand utreda säkerhetsincidenter.
- **Health Checks:** Vår pipeline verifierar applikationens hälsa (/health) innan en deployment slutförs, vilket förhindrar att trasig kod tar ner tjänsten.

Sammanfattning

Syftet med detta projekt har varit att bygga en digital infrastruktur till en onlinevårdcentral, Northstar Care. Fokuset har legat på att skapa ett nätverk som är både enkelt att hantera och har rum för expansion med säkerhetsaspekterna i åtanke.

Skapandet av resurser i Azure

Azure App Service funkar som systemets motor där själva programkoden körs. För att kunna hämta och lagra patientfiler säkert är den sammankopplad med ett storage account. Istället för att skriva in lösenord i koden, som skapar säkerhetsbrister, så används miljövariabler. Dessa funkar som instruktioner som injiceras direkt in i app-service vid start, vilket gör att applikationen vet exakt hur den ska kommunicera med lagringskonto utan att avslöja säkerhetsnycklar i källkoden.

Pipeline

- Pipelinen vi har byggt var en viktig del av projektet. Den används inte bara som en byggprocess för att deploya projektet från DevOps till vår Azure subscription med hjälp av en Service Connection, utan den används också som ett tidigt och kontinuerligt kontrollerat säkerhetsflöde. Genom de olika säkerhetsscans som lagts till under hela processen säkerställer vi att risker och sårbarheter som upptäcktes hanteras innan koden deploys i produktion.

Automatiserad säkerhetsgranskning

- Vad vi har gjort: Vi har gjort så att varje ny ändring i portalen måste passera automatiserade säkerhetskontroller innan den släpps ut. Systemet letar på egen hand efter svagheter i koden, läckta lösenord och andra brister i de externa verktygen som vi använder.
- Varför detta är viktigt: Människor gör misstag och säkerhetshot utvecklas snabbt. Genom att automatisera dessa tester bygger vi ett skydd som aktivt stoppar säkerhetsbrister från att nå portalen. Detta gör att vi kan arbeta mer effektivt utan att dra ner på säkerheten.

Datahantering och driftsäkerhet

- Vad vi har gjort: För att skydda patientdata har vi utöver de automatiserade säkerhetsgranskningarna också infört automatiserade städrutiner som raderar gamla dokument efter 365 dagar. Vi har också lagt till utökad övervakning "loggning" och infört en hälsokontroll i systemet som känner efter att portalen fungerar som den ska innan en uppdatering släpps igenom.
- Varför detta är viktigt: Att spara data längre än nödvändigt är en onödig risk och med automatisk rensning följer vi principen om dataminimering och minskar risken vid eventuella dataläckor. Hälsokontrollerna och övervakningen säkerställer i sin tur att trasig kod aldrig kan krascha hela portalen. Resultatet är en stabil tjänst man kan

Återstående problem

- Hårdkodad secret: Hårdkodade reservlösenord i källkoden skapar en kritisk säkerhetsbrist. Lösenordet är sparat i källkoden för vem som helst att se och för den kunniga att använda.
- Varningar i Defender: För webbappen: Managed identity och Diagnostic logs. För Lagring Kontot: Public exposure och Shared keys.
- Dependencies: Vi använder äldre versioner av tredjepartsbibliotek vilket kan leda till en öppning för en angripare.
- Avsaknad av Rate-limiting och input validering: Vi skyddar inte mot brute-force eller traverse-attacker.

Slutsats

Genom att implementera en automatiserad DevSecOps-pipeline i Azure har Northstar Care gått från en reaktivt till en proaktivt applikation. Införandet av automatiserade säkerhetskontroller, systemtilldelade identiteter och en uppgraderad servermiljö säkerställer nu en stabil och skyddad plattform. Även om några kod och konfigurationsbrister kvarstår att åtgärda innan applikationen publiceras på riktigt så har projektet stärkt systemets övergripande säkerhet och tillgänglighet.

Personlig reflektion och fördjupning – Albin Sulejmani

Det svåraste var utan tvekan identitets- och behörighetshanteringen i Azure. Problemen med olika domäner (@learn.se vs @azurenodhill.onmicrosoft.com) ledde till att gruppmedlemmar fick olika roller och att vissa inte alls fick tillgång till resurser vi behövde. Vi stötte på scenarier där gruppmedlemmar hade olika roller (Contributor vs. Admin), vilket hindrade oss från att utföra kritiska uppgifter som att committa till main eller hantera resurser.

Det är lätt att underskatta hur komplex hybrididentitetshantering är i Azure. Det räcker inte att "lägga till någon i projektet", utan roller, prenumerationer och domänkopplingar måste stämma överens på flera nivåer samtidigt. Detta kostade oss betydande tid som annars hade kunnat läggas på säkerhetsförbättringar i koden.

Det mest värdefulla var insikten om DevSecOps som ett tankesätt snarare än en verktygsuppsättning. Att integrera SAST, secrets scanning, DAST och dependency scanning, den tekniska aspekten i pipelinen, var relativt enkel och det lärorika var förståelsen för varför dessa gates måste sitta i CI/CD-flödet och inte utföras manuellt i efterhand.

När vi testade att skicka en payload på 1 000 000 tecken mot /api/assistant/ask och fick en HTTP 500, blev det tydligt på ett konkret sätt att säkerhet inte är något man "lägger på" i slutet, utan något som måste byggas in från grunden i varje lager. Det visade att vi saknar tidig input-validering och rate-limiting, vilket i en molnmiljö inte bara är en teknisk risk utan en affärsmässig sådan. Input-validering, rate-limiting och resursbegränsningar tillhör applikationens grundläggande robusthet, inte en säkerhetspatch.

Om jag byggde lösningen från grunde är dessa tre punkter vad jag lagt fokus på att förbättra

- Inga hårdkodade fallback-värden överhuvudtaget. Alla hemligheter hanteras via Azure Key Vault, med Managed Identity som enda autentiseringsmetod mot känsliga resurser, alltså ingen connection string i miljövariabler.
- Input-validering och rate-limiting redan i första commit. Att initialt inkludera express-validator och express-rate-limit är en liten tidsinvestering som eliminerar en hel kategori av potentiella risker (brute-force, payload flooding, trust boundary violations i casesService.js) innan de ens uppstår.
- Tydligare rollstruktur i Azure från start. Istället för att lösa behörighetsproblem reaktivt när de uppstår, hade vi skapat en dokumenterad RBAC-matris i förväg och verifierat att alla gruppmedlemmar har korrekt åtkomst via rätt domän innan något annat arbete påbörjades.

Personlig fördjupning – Albin Sulejmani

En av de mest kritiska riskerna vi identifierade i projektet var det hårdkodade fallback värdet (hemligheten) i server/index.js:

```
const JWT_SECRET = process.env.JWT_SECRET || "northstar-dev-secret";
```

I dokumentationen konstaterar vi att detta är en allvarlig säkerhetsrisk, men det är värt att dyka djupare in på varför och vad konsekvenserna faktiskt kan innebära.

En JWT (JSON Web Token) består av tre delar: header, payload och signatur. Signaturen skapas genom att hasha header + payload med JWT_SECRET. Servern verifierar inkommande token genom att göra samma uträkning och jämföra resultaten. Om hemligheten är känd (publik) kan vem som helst skapa en godtycklig payload. Exempel: { "role": "admin", "username": "angripare" } och signera den med "northstar-dev-secret" och få servern att acceptera den som en giltig autentiserad session. Detta leder inte bara till dataintrång, utan det fullständigt kringgår hela autentiseringslagret.

Detta är ett typexempel på en "fail-open" design. Det innebär att om systemet misslyckas med att hämta den säkra miljövariabeln från Azure, faller det tillbaka på en hårdkodad och känd hemlighet istället för att neka åtkomst. För att uppnå en "fail-safe" arkitektur (vilket jag föreslår som förbättring) bör applikationen istället programmeras att krascha omedelbart vid start om hemligheten saknas. Det är alltid säkrare att en tjänst är tillfälligt nere än att den körs med en publikt känd säkerhetsnyckel.

Under vår säkerhetsanalys utförde vi ett belastningstest mot applikationen. När vi skickade en ohanterad payload på 1 000 000 tecken resulterade det i ett HTTP 500-fel och extrem latens. Detta visade en allvarlig sårbarhet för resursutmattning (Resource Exhaustion).

I en traditionell servermiljö hade detta "bara" inneburit att servern blivit överbelastad och oåtkomlig. Men i en molnmiljö som Azure förändras risken och en ny kritisk attackvektor introduceras: Denial of Wallet (DoW).

DoW utnyttjar molnets grundläggande affärsmodell, det vill säga att man betalar för förbrukade resurser såsom CPU, minne, bandbredd och antal instanser. Angriparens primära mål är därmed inte nödvändigtvis att sänka tjänsten, utan att tvinga systemet att förbruka extrema, ohållbara mängder resurser som resulterar i extrema kostnader för företaget.

Om vår applikation hade haft automatisk skalning (auto-scaling) aktiverad för att garantera hög tillgänglighet, hade Azure omedelbart startat nya instanser för att försöka hantera den tunga, skadliga trafiken. Eftersom applikationen i dagsläget saknar både rate-limiting och tidig input-validering (t.ex. en gräns för payload-storlek innan anropet behandlas), skulle systemet acceptera och försöka bearbeta varje anrop.

En angripare skulle därmed kunna orsaka massiv ekonomisk skada för Northstar Care utan att ens behöva ta ner plattformen. Detta framför en viktig DevSecOps-lärdom: i molnet är säkerhetsåtgärder som input-validering, resursbegränsningar och kvothantering inte enbart tekniska skyddsmekanismer, utan de är helt avgörande för verksamhetens ekonomi.

Slutsats för personlig fördjupning

Genom att kombinera och integrera flerlagrat försvar (Defense in Depth) med fail-safe, input-validering och hårdkodade hemligheter med molnspecifika skydd (Managed Identity, Web Application Firewall) skapar vi en robust infrastruktur som skyddar både applikationen och användares integritet och verksamhetens ekonomi.